

INF1022 - Analisadores Léxicos e Sintáticos

Expressões Regulares e Linguagens Regulares

Edward Hermann Haeusler Jefferson Santos

PUC-Rio

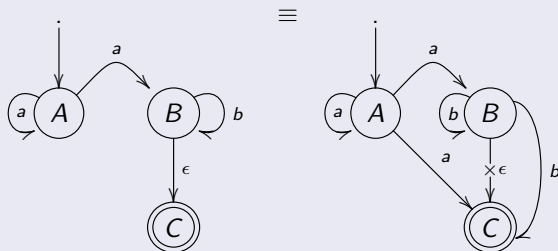
Departamento de Informática

Linguagens Regulares

- Uma Linguagem L é Regular, se e somente se, existe um AFD $\mathcal{A}$, tal que, $L(\mathcal{A}) = L$.
- Uma Linguagem L é Regular, se e somente se, existe um AFND $\mathcal{A}$, tal que, $L(\mathcal{A}) = L$.
- Uma Linguagem L é Regular, se e somente se, existe uma expressão regular e tal que $L(e) = L$.

Autômatos com transições ϵ

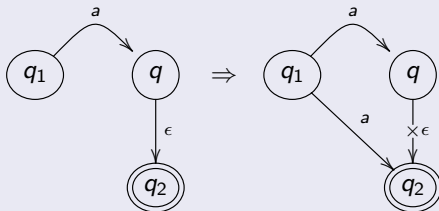
Uma conveniência formal



Autômatos com transições ϵ

Eiminando transições ϵ

Para cada $q_2 \in \delta(q, \epsilon)$ e $q \in \delta(q_1, a)$, faz-se $q_2 \in \delta(q_1, a)$. Eliminar a transição ϵ quando todos os pares (q_1, q_2) forem considerados



Repita esta transformação até não haver mais transições ϵ .

Abrangência de ϵ -Autômatos

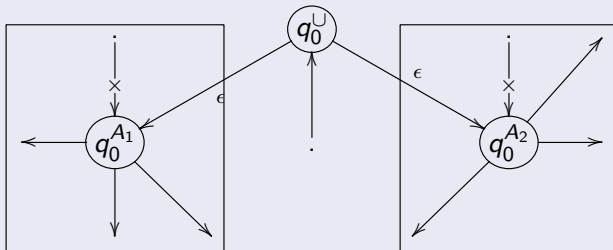
Teorema Seja $\mathcal{A}$ um ϵ -Autômato e $\mathcal{A}^\epsilon$ o autômato resultante da eliminação de transições ϵ de $\mathcal{A}$. Então $L(\mathcal{A}) = L(\mathcal{A}^\epsilon)$.

Ainda não estamos prontos para entender, em detalhe, o enunciado acima. Obs: $\mathcal{A}^\epsilon$ pode não ser um AFD. Por que ??

Apesar de não aparecerem explicitamente como não-determinísticos, os autômatos com transição ϵ , de fato, são não-determinísticos.

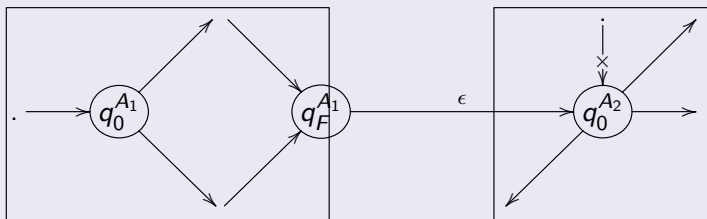
Definindo autômatos a partir de autômatos

Um autômato que aceita $L(A_1) \cup L(A_2)$



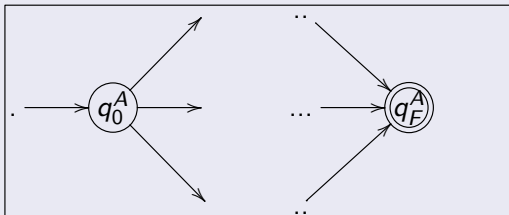
Definindo autômatos a partir de autômatos

Um autômato que aceita $L(A_1).L(A_2)$



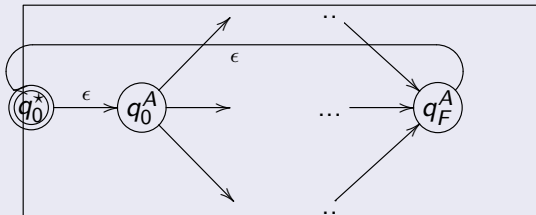
Definindo autômatos a partir de autômatos

O autômato abaixo aceita $L(A)$



Definindo autômatos a partir de autômatos

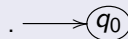
Um autômato que aceita $L(A)^*$



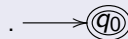
Definindo autômatos a partir de autômatos

Autômatos Básicos

Linguagem Vazia ($\emptyset$)



$\{\epsilon\}$



Linguagem $\{\sigma\}$



Expressões Regulares e seus Autômatos

Expressão Regular

- “ $\emptyset$ ”, “ ϵ ” e “ a ”, para todo $a \in \Sigma$ são expressões regulares (E.R.).
- Se “ e_1 ” e “ e_2 ” são expressões regulares, então “ $e_1 + e_2$ ”, “ $e_1.e_2$ ” e “ e_1^* ” são E.R.s

Expressões Regulares e seus Autômatos

Expressão Regular

- “ $\emptyset$ ”, “ ϵ ” e “ a ”, para todo $a \in \Sigma$ são expressões regulares (E.R.).
- Se “ e_1 ” e “ e_2 ” são expressões regulares, então “ $e_1 + e_2$ ”, “ $e_1.e_2$ ” e “ e_1^* ” são E.R.s

Linguagem de uma E.R. , $\mathcal{L}(e)$

- $\mathcal{L}(\epsilon) = \{\epsilon\}$, $\mathcal{L}(\emptyset) = \emptyset$ e $\mathcal{L}(\sigma) = \{\sigma\}$.
- $\mathcal{L}(e_1.e_2) = \mathcal{L}(e_1).\mathcal{L}(e_2)$,
- $\mathcal{L}(e_1 + e_2) = \mathcal{L}(e_1) \cup \mathcal{L}(e_2)$
- $\mathcal{L}(e_1^*) = \mathcal{L}(e_1)^*$
- $\mathcal{L}(e_1^+) = \mathcal{L}(e_1)^* - \{\epsilon\}$

Expressões Regulares e seus Autômatos

$\emptyset a = a \emptyset = \emptyset$	$\varepsilon a = a \varepsilon = a$
$\emptyset^* = \varepsilon$	$\varepsilon^* = \varepsilon$
$a \cup \emptyset = a$	$a^* = (a^*)^*$
$a(b \cup c) = ab \cup ac$	$(b \cup c)a = ba \cup ca$
$(ab)^*a = a(ba)^*$	$(a \cup b)^* = (a^* \cup b)^* = a^*(a \cup b)^* = (a \cup ba^*)^* = (a^*b^*)^* = a^*(ba^*)^*$

E.R. e Aut. Finitos

$[E.R. \subseteq AFND]$ Se L é descrita por uma E.R. então L é aceita por Aut Finito.

E.R. e Aut. Finitos

$[E.R. \subseteq AFND]$ Se L é descrita por uma E.R. então L é aceita por Aut Finito. $[AFD \subseteq E.R.]$ Se L é aceita por Aut. Finito então L é descrita por E.R.

Expressões Regulares como Ferramenta para *Pattern-Matching*

REGULAR EXPRESSIONS



Pattern-Matching e o padrão POSIX

- Literal Characters
- Character Classes
- Shorthand Character Classes / 'dot'
- Non-Printable Characters / Anchors
- Quantifiers
- Modifiers

Pattern-Matching e o padrão POSIX

Are simple characters the way you expect them.

e.g. `/a/` -> will match 'a' in a text

Pattern-Matching e o padrão POSIX

`\t` -> tab character (ASCII 0x09)
`\r` -> carriage return (0x0D)
`\n` -> line feed (0x0A)
`(\a (bell, 0x07), \e (escape, 0x1B), \f (form feed, 0x0C) ,\v (vertical tab, 0x0B) )`

`\xFF` -> hexadecimal index in the char. set
e.g. `\xA9` -> copyright symbol in the Latin-1

`\uFFFF` -> Unicode character
e.g. `\u20AC` -> the euro currency sign

`^` begin of the string

`$` end of the string

`\b` -> word boundary - `\B` -> not word boundary

Pattern-Matching e o padrão POSIX

- Groups of character
[abcdefgh0123]
- Range of character
[a-h0-3]
- Inverse Range of Character
[^i-z4-9]

e.g. `/[gG]uide[IL]ine/` -> guideline or
guideLine

Pattern-Matching e o padrão POSIX

```
\d -> [0-9]  
\w -> [a-zA-Z]  
\s -> whitespace + [\t\r\n]
```

negative:

```
\D -> [^0-9]  
\W -> [^a-zA-Z]  
\S
```

```
'.' -> any character!!
```

$$AFD \subseteq E.R.$$

[$AFD \subseteq E.R.$] Se L é aceita por Aut. Finito então L é descrita por E.R.

Três métodos diferentes para provar este teorema:

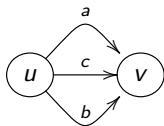
- Decomposição de autômatos, e;
- Sistema de equações (Lema de Arden), e;
- Eliminação de estados;

Decomposição de um Autômato $\mathcal{A}$

α_{uv}^X é a linguagem de todas as palavras que estão associadas a um caminho em $\mathcal{A}$, começando no estado u , indo até v , passando somente por estados em X .

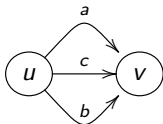
Definindo $\alpha_{u,v}^X$ recursivamente (I)

- $\alpha_{u,v}^{\emptyset} = \sum_{\sigma \in \{\sigma: \delta(u, \sigma) = v\}} \sigma$. Exemplo, $\alpha_{u,v}^{\emptyset} = (a + b + c)$, para:



Definindo $\alpha_{u,v}^X$ recursivamente (I)

- $\alpha_{u,v}^{\emptyset} = \sum_{\sigma \in \{\sigma: \delta(u, \sigma) = v\}} \sigma$. Exemplo, $\alpha_{u,v}^{\emptyset} = (a + b + c)$, para:



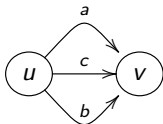
- $\alpha_{u,u}^{\emptyset} = \epsilon + \sum_{\sigma \in \{\sigma: u \in \delta(u, \sigma)\}} \sigma$. Exemplo $\alpha_{u,u}^{\emptyset} = (\epsilon + a + b)$, para:



O conjunto das palavras que vão de u para u sem passar por nenhum outro estado inclui o ϵ !!

Definindo $\alpha_{u,v}^X$ recursivamente (I)

- $\alpha_{u,v}^{\emptyset} = \sum_{\sigma \in \{\sigma: \delta(u, \sigma) = v\}} \sigma$. Exemplo, $\alpha_{u,v}^{\emptyset} = (a + b + c)$, para:



- $\alpha_{u,u}^{\emptyset} = \epsilon + \sum_{\sigma \in \{\sigma: u \in \delta(u, \sigma)\}} \sigma$. Exemplo $\alpha_{u,u}^{\emptyset} = (\epsilon + a + b)$, para:

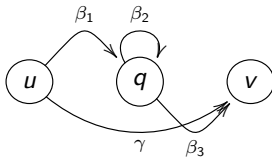


O conjunto das palavras que vão de u para u sem passar por nenhum outro estado inclui o ϵ !!

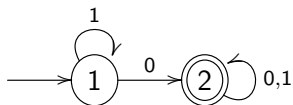
- Se para todo $\sigma, \delta(u, \sigma) \uparrow$ então $\alpha_{u,v}^{\emptyset} = \emptyset$.
- Se para todo $\sigma, \delta(u, \sigma) \uparrow$ então $\alpha_{u,u}^{\emptyset} = \emptyset + \epsilon = \epsilon$.

Definindo $\alpha_{u,v}^X$ recursivamente (II)

- $\alpha_{uv}^X = \alpha_{uv}^{X-q} + \alpha_{uq}^{X-q} . (\alpha_{qq}^{X-q})^* . \alpha_{qv}^{X-q}$, para algum $q \in X$.



Exemplo



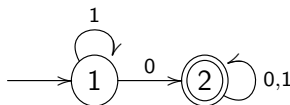
$$\alpha_{11}^{\emptyset} = \epsilon + 1$$

$$\alpha_{12}^{\emptyset} = 0$$

$$\alpha_{21}^{\emptyset} = \emptyset$$

$$\alpha_{22}^{\emptyset} = (\epsilon + 0 + 1)$$

Avançando na recursão



Escolhendo $q = 1$ a recursão fica sendo $\alpha_{ij}^{\{1\}} = \alpha_{ij}^{\emptyset} + \alpha_{i1}^{\emptyset}(\alpha_{11}^{\emptyset})^* \alpha_{1j}^{\emptyset}$, temos

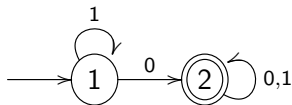
$$\alpha_{11}^{\{1\}} = (\epsilon + 1) + (\epsilon + 1)(\epsilon + 1)^*(\epsilon + 1)$$

$$\alpha_{12}^{\{1\}} = 0 + (\epsilon + 1)(\epsilon + 1)^*0$$

$$\alpha_{21}^{\{1\}} = \emptyset + \emptyset(\epsilon + 1)^*(\epsilon + 1)$$

$$\alpha_{22}^{\{1\}} = (\epsilon + 0 + 1) + \emptyset(\epsilon + 1)^*0$$

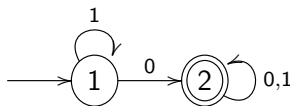
Simplificando expressões no exemplo



Escolhendo $q = 1$ a recursão fica sendo $\alpha_{ij}^{\{1\}} = \alpha_{ij}^{\emptyset} + \alpha_{i1}^{\emptyset}(\alpha_{11}^{\emptyset})^* \alpha_{1j}^{\emptyset}$, temos

$\alpha_{11}^{\{1\}}$	$= (\epsilon + 1) + (\epsilon + 1)(\epsilon + 1)^*(\epsilon + 1)$	1^*
$\alpha_{12}^{\{1\}}$	$= 0 + (\epsilon + 1)(\epsilon + 1)^*0$	1^*0
$\alpha_{21}^{\{1\}}$	$= \emptyset + \emptyset(\epsilon + 1)^*(\epsilon + 1)$	$\emptyset$
$\alpha_{22}^{\{1\}}$	$= (\epsilon + 0 + 1) + \emptyset(\epsilon + 1)^*0$	$\epsilon + 0 + 1$

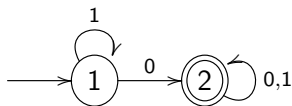
Avançando na recursão (II)



Escolhendo $q = 2$ a recursão fica sendo $\alpha_{ij}^{\{1,2\}} = \alpha_{ij}^{\{1\}} + \alpha_{i2}^{\{1\}}(\alpha_{22}^{\{1\}})^* \alpha_{2j}^{\{1\}}$, temos

$$\begin{aligned}
 \alpha_{11}^{\{1,2\}} &= 1^* + (1^*0)(\epsilon + 0 + 1)^*\emptyset \\
 \alpha_{12}^{\{1,2\}} &= 1^*0 + (1^*0)(\epsilon + 0 + 1)^*(\epsilon + 0 + 1) \\
 \alpha_{21}^{\{1,2\}} &= \emptyset + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*\emptyset \\
 \alpha_{22}^{\{1,2\}} &= (\epsilon + 0 + 1) + (\epsilon + 0 + 1)(\epsilon + 1)^*(\epsilon + 0 + 1)
 \end{aligned}$$

Avançando na recursão (II): simplificando



Escolhendo $q = 2$ a recursão fica sendo $\alpha_{ij}^{\{1,2\}} = \alpha_{ij}^{\{1\}} + \alpha_{i2}^{\{1\}}(\alpha_{22}^{\{1\}})^* \alpha_{2j}^{\{1\}}$, temos

$\alpha_{11}^{\{1,2\}}$	$= 1^* + (1^*0)(\epsilon + 0 + 1)^*\emptyset$	1^*
$\alpha_{12}^{\{1,2\}}$	$= 1^*0 + (1^*0)(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$1^*0(0 + 1)^*$
$\alpha_{21}^{\{1,2\}}$	$= \emptyset + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*\emptyset$	$\emptyset$
$\alpha_{22}^{\{1,2\}}$	$= (\epsilon + 0 + 1) + (\epsilon + 0 + 1)(\epsilon + 1)^*(\epsilon + 0 + 1)$	$(0 + 1)^*$

Uma E.R. para a linguagem do autômato $\mathcal{A}$

$$\mathcal{L}(A) = \sum_{q_f \in F} \alpha_{q_0 q_f}^Q$$

No exemplo $\mathcal{L}(A) = 1^*0(0 + 1)^*$

O Lema de Arden (I)

Lemma (Arden)

*Sejam A e B linguagens formais e seja a equação $X = AX + B$.
Então a única solução desta equação é $X = A^*B$*

Prova: (1) $A(A^*B) + B = A^+B + B = (A^+ + \epsilon)B = A^*B$

O Lema de Arden (II)

Lemma (Arden a direita)

*Sejam A, B linguagens formais e seja a equação $X = AX + B$. Se $\epsilon \notin A$ então a única solução desta equação é $X = A^*B$*

Prova: (2) Unicidade: Seja X outra solução para a equação: $B \subseteq X$, daí $AB \subseteq AX \subseteq X$. Temos também que $A(AB) \subseteq A(AX) \subseteq AX \subseteq X$, portanto $A^*B \subseteq X$. Então $X = A^*B + C$ com $A^*B \cap C \neq \emptyset$. Como X é solução da equação: $A^*B + C = A(A^*B + C) + B = A^*B + AC$. Temos $C = C \cap AC$, impossível, pois $\epsilon \notin A$. Pense na palavra mais curta de C !!!

Lemma (Arden a esquerda)

Sejam A, B linguagens formais e seja a equação $X = XA + B$. Se $\epsilon \notin A$ então a única solução desta equação é $X = BA^$*

Linguagem a esquerda de um estado

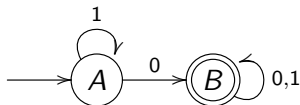
Seja $\mathcal{A} = \langle Q, \Sigma, q_0, F, \delta \rangle$ um autômato finito. Dado um estado $q \in Q$, define-se a linguagem a esquerda de q como sendo:

$$L_q = \{\omega \in \Sigma^* : \hat{\delta}(q_0, \omega) = q\}$$

ou seja, L_q é o conjunto de palavras que “alcançam” o estado q desde que lidas a partir de q_0 .

Sistema de equações e Aut. Finitos

Seja $\mathcal{A}$ o autômato abaixo:



O sistema de equações abaixo representa as relações entre as linguagens esquerdas L_q definidas pelas transições em cada estado q

$$\begin{aligned} L_A &= L_A 1 + \epsilon \\ L_B &= L_A 0 + L_B (0 + 1) \end{aligned}$$

Resolvendo o sistema de equações, teremos $L_B = L(\mathcal{A})$

Resolvendo o sistema

$$\begin{aligned}L_A &= \epsilon + L_A 1 \\L_B &= L_A 0 + L_B(0 + 1)\end{aligned}$$

Pelo lema de Arden a esquerda, $L_A = \{\epsilon\}\{1\}^* = \{1\}^*$, descrita por 1^* .

Então $L_B = L_A 0 + L_B(0 + 1) = 1^*0 + L_B(0 + 1)$, por Arden temos

$$L_B = (1^*0)(0 + 1)^* = 1^*0(0 + 1)^*$$

Resolvendo o sistema

$$\begin{aligned}L_A &= \epsilon + L_A 1 \\L_B &= L_A 0 + L_B(0 + 1)\end{aligned}$$

Pelo lema de Arden a esquerda, $L_A = \{\epsilon\}\{1\}^* = \{1\}^*$, descrita por 1^* .

Então $L_B = L_A 0 + L_B(0 + 1) = 1^*0 + L_B(0 + 1)$, por Arden temos

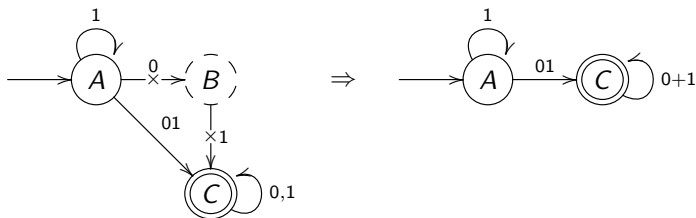
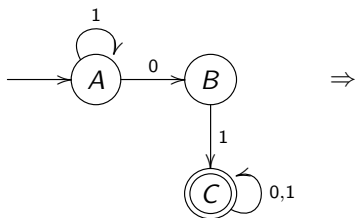
$$L_B = (1^*0)(0 + 1)^* = 1^*0(0 + 1)^*$$

Método de eliminação de estados

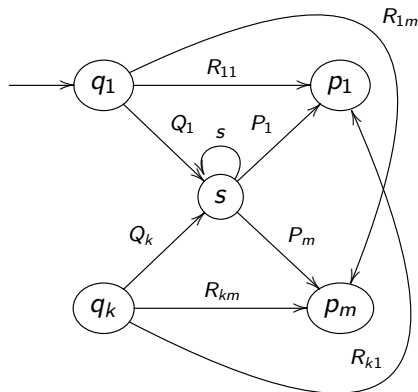
- O algoritmo da eliminação de estados permite obter uma expressão regular que representa a linguagem reconhecida por um AF.
- Ele consiste em remover, em cada passo, um estado do autômato finito de forma que a linguagem reconhecida não se altere.
- A cada passo do método, em que se elimina um estado, é produzido um AF em que suas transições podem conter expressões regulares.
- O algoritmo termina quando não existirem mais estados não iniciais ou não finais para eliminar.

Método de eliminação de estados

A intuição sobre a eliminação de estados: (Retirando o estado B)



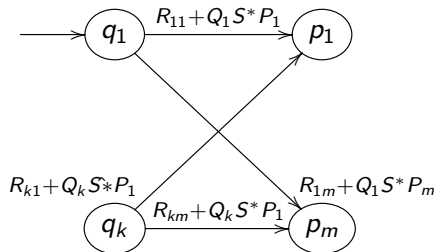
Processo de eliminação de um estado (estado S)



- Supomos que o estado s possui os predecessores $q_1, q_2, \dots, q_k$ e sucessores $p_1, p_2, \dots, p_m$.
- Pode-se criar uma ER para cada transição de um predecessor para S e de S para um estado sucessor.
- Existe uma ER R_{ij} que sai de um estado q para um estado p para todo i e j .
- Note que todos esses arcos podem não existir, neste caso podemos criar um com a transição $\emptyset$.

Processo de eliminação de um estado (estado S)

Abaixo está o resultado de quando se elimina o estado s .



- Todos os arcos que passam por s são excluídos.
- Para compensar, é introduzido uma ER para cada predecessor q_i de s e sucessor p_j de s que representa todos os caminhos que iniciam em q_i , passam por s (incluindo loops em s) e chegam em p_j .
- O resultado dessa ER é $Q_iS^*P_j$, e essa ER é adicionada com o operador de união no arco $q_i \rightarrow p_j$.

Método de eliminação de estados

Resumo do algoritmo

- 1 Se houver mais de um estado final, adicione um novo estado final, torne todos os outros estados como não finais e adicione uma transição ϵ que tem como origem cada um dos antigos estados finais e como destino o novo estado final.
- 2 Para cada estado que não seja final e nem inicial, elimine-os aplicando o processo apresentado nos slides acima e atualize as transições de acordo.
- 3 Note que quando o estado inicial for o único estado final, ao final do processo de eliminação de estados, vamos ter um autômato com somente o estado inicial. Caso contrário, teremos sempre um autômato com dois estados, o inicial e o final.